

Análise Técnica Aprofundada sobre Simulação de Interfaces de Rede Wi-Fi e LTE em Ambientes de Virtualização Android (Redroid)

Resumo Executivo

A virtualização de sistemas operacionais móveis em infraestruturas de nuvem e contêineres representa um dos desafios técnicos mais sofisticados da computação moderna. O projeto **redroid** (Remote Android), uma implementação de Android em contêineres Docker (AIC - Android In Container), destaca-se pela eficiência ao compartilhar o kernel do sistema hospedeiro, eliminando a sobrecarga da emulação de hardware tradicional. No entanto, essa arquitetura de kernel compartilhado introduz limitações críticas na emulação de subsistemas de rádio, especificamente Wi-Fi (WLAN) e redes celulares (LTE/5G). A presente pesquisa investiga a existência de imagens pré-configuradas e metodologias para superar essas limitações, respondendo à demanda por simulação de conectividade sem fio em ambientes onde a interface nativa é, invariavelmente, Ethernet.

A análise exaustiva dos repositórios de desenvolvimento, documentação técnica e discussões da comunidade de engenharia revela que **não existe uma imagem monolítica oficial do redroid capaz de simular Wi-Fi ou LTE de forma autônoma e imediata ("out-of-the-box")**. Isso ocorre porque a abstração de hardware de rede no redroid depende intrinsecamente das capacidades e módulos carregados no kernel do sistema operacional *host* (Linux), uma camada que reside fora do controle direto da imagem do contêiner Android.

Contudo, a pesquisa identificou e validou dois vetores primários de solução que permitem alcançar a simulação desejada:

1. **Emulação de Hardware Virtual (Kernel-Space):** A utilização de módulos de kernel como `virt_wifi` (para Android 12/13+) e `mac80211_hwsim` (para versões legadas). Esta abordagem cria interfaces de rede sem fio reais (`wlan0`) visíveis ao sistema, mas exige configuração complexa no host e gerenciamento de namespaces de rede, além de apresentar riscos de instabilidade na conexão de depuração (ADB).
2. **Instrumentação Dinâmica de Runtime (User-Space):** A aplicação de técnicas de *API Hooking* através de frameworks como Xposed ou LSPosed, executados sobre o Magisk. Esta é a solução mais viável para a "simulação de percepção", onde o sistema operacional e os aplicativos são induzidos a acreditar que estão operando sobre Wi-Fi ou LTE, embora o transporte de dados ocorra via Ethernet.

Este relatório detalha a arquitetura, implementação e limitações de ambas as abordagens, fornecendo um guia técnico para engenheiros e pesquisadores que necessitam replicar condições de rede móvel em instâncias redroid.

1. Fundamentos da Virtualização de Conectividade no Android

Para compreender a complexidade da solicitação — "uma imagem que simule Wi-Fi ou LTE" — é imperativo estabelecer primeiro as diferenças fundamentais entre a arquitetura de rede do Android em dispositivos físicos e sua implementação em ambientes de contêineres.

1.1 O Paradigma do Redroid vs. Emuladores Tradicionais

O Android Emulator tradicional (baseado em QEMU/Goldfish) emula uma máquina completa, incluindo barramentos de hardware e periféricos fictícios. Nesse cenário, o emulador pode apresentar um "modem virtual" ao sistema operacional convidado, permitindo a simulação trivial de estados de telefonia e Wi-Fi através de comandos de console (ex: network speed lte).¹

Em contraste, o redroid opera como um contêiner leve. Ele não virtualiza o hardware; ele compartilha os recursos do kernel do host Linux. A interface de rede padrão de um contêiner Docker é uma ponte (bridge) virtual conectada à interface física do host. Para o Android rodando dentro do redroid, essa interface é apresentada e detectada nativamente como Ethernet (eth0).³ O *Android Open Source Project* (AOSP) possui pilhas de protocolo distintas para Ethernet e Wi-Fi. A pilha Ethernet é tratada como uma conexão cabeada, estática e "ilimitada", enquanto a pilha Wi-Fi envolve varredura (scanning), autenticação (WPA/WPA2) e gerenciamento de energia.

1.2 A Necessidade de Simulação

A demanda por simulação de Wi-Fi ou LTE no redroid não é meramente estética, mas funcional. Inúmeras aplicações Android modernas implementam lógicas de verificação de conectividade rigorosas:

- **Restrições de Download:** Jogos e lojas de aplicativos frequentemente exigem Wi-Fi para downloads de grandes arquivos (OBB assets) para evitar consumo excessivo de dados móveis, recusando-se a proceder em conexões Ethernet ou "desconhecidas".
- **Identificação de Dispositivo:** Aplicações bancárias e de streaming (VOD) verificam o tipo de rede para avaliar riscos de segurança ou ajustar a qualidade do bitrate. A ausência de uma interface Wi-Fi ou Celular pode ser interpretada como um ambiente de emulação ou "root", ativando medidas anti-fraude.⁴
- **Protocolos Específicos:** Algumas aplicações dependem de protocolos de descoberta

de rede local (como mDNS ou UPnP) que, em suas implementações Android, escutam especificamente nas interfaces wlan0 e ignoram eth0.

Portanto, a "imagem" procurada pelo usuário deve ser capaz de preencher o hiato entre a infraestrutura real (Ethernet do Servidor) e a expectativa do software (Wi-Fi/LTE do Dispositivo Móvel).

2. Arquitetura de Conectividade do Android e Pontos de Interceptação

A análise aprofundada do funcionamento interno do Android revela por que a simples instalação de uma imagem não resolve o problema sem configurações adicionais. O subsistema de conectividade do Android é hierárquico, e a simulação pode ocorrer em diferentes níveis dessa hierarquia.

2.1 O Papel do ConnectivityManager

No topo da hierarquia reside o ConnectivityManager, uma API de sistema que fornece informações sobre o estado da rede para as aplicações. Quando um aplicativo solicita informações de rede, o sistema verifica os objetos NetworkCapabilities e NetworkInfo (este último, embora obsoleto em APIs recentes, ainda é amplamente utilizado).

Em uma instância padrão do redroid:

- A interface física é eth0.
- O ConnectivityService detecta o link Ethernet.
- O transporte reportado é TRANSPORT_ETHERNET.
- As APIs retornam TYPE_ETHERNET.

Para simular LTE ou Wi-Fi, é necessário alterar esse fluxo. Se a alteração for feita no nível da API (User-Space), o sistema mente para o aplicativo. Se a alteração for feita no nível do Kernel (Kernel-Space), o sistema realmente enxerga uma interface sem fio.

2.2 Camada de Abstração de Hardware (HAL)

Abaixo do framework Java, o Android utiliza HALs (wifi.hal, radio.hal) para se comunicar com o kernel.

- **Wi-Fi:** O daemon wpa_supplicant (ou wificond nas versões modernas) espera encontrar uma interface de rede sem fio compatível com as extensões sem fio do Linux (nl80211). Se o kernel do host não fornecer uma interface que responda a esses comandos, o subsistema Wi-Fi do Android falha ao iniciar, e o botão de "Ligar Wi-Fi" na interface gráfica retorna imediatamente para a posição "Desligado".

- **LTE/Rádio:** A emulação de LTE é ainda mais complexa, pois envolve a RIL (Radio Interface Layer). O redroid, rodando em servidores x86 ou ARM sem modems baseband, não possui hardware para interagir com a RIL. A simulação de LTE, portanto, é quase exclusivamente realizada via software (API Hooking) ou através de propriedades de sistema (build.prop) que definem a presença de rádio, embora sem funcionalidade real de chamadas.⁵

3. Solução 1: Simulação Nativa via Módulos de Kernel (A Abordagem "Hard")

A pesquisa nos repositórios técnicos do projeto redroid e discussões de desenvolvimento de kernel Linux aponta que a simulação real de uma interface Wi-Fi é possível, mas depende estritamente de módulos de kernel carregados no *host*. Isso invalida a existência de uma "imagem Docker mágica" que funcione isoladamente, pois contêineres não podem carregar módulos de kernel por razões de segurança e arquitetura.

3.1 O Legado: mac80211_hwsim

Historicamente, a comunidade utilizava o módulo `mac80211_hwsim` para simular rádios Wi-Fi. Este módulo do kernel Linux cria interfaces de rede virtuais que se comportam como rádios 802.11 reais, permitindo a criação de redes mesh ou pontos de acesso virtuais puramente em software.

Mecanismo de Implementação

A implementação envolvia carregar o módulo no host (`modprobe mac80211_hwsim`), criando interfaces `wlan0` e `wlan1`. Em seguida, scripts complexos moviam uma dessas interfaces para o *namespace* de rede do contêiner redroid. Dentro do Android, o serviço `wpa_supplicant` podia então gerenciar essa interface, permitindo que o Android "conectasse" a um AP virtual simulado pela segunda interface.³

A Obsolescência no Android 13+

Relatórios técnicos e *issues* no GitHub do redroid indicam que esse método foi efetivamente abandonado ou tornou-se inviável a partir do Android 13 (AOSP 13). As mudanças na pilha de Wi-Fi do Android e na maneira como o Netlink (protocolo de comunicação kernel-usuário) opera tornaram a integração com o `mac80211_hwsim` extremamente instável.³ Desenvolvedores principais do projeto, como zhouziyang, confirmaram que embora teoricamente possível, a complexidade de manutenção superou os benefícios para a maioria dos casos de uso, levando à busca por alternativas mais modernas.

3.2 O Estado da Arte: Módulo `virt_wifi`

A solução contemporânea recomendada para simulação de Wi-Fi em ambientes redroid,

especialmente para versões AOSP 12 e superiores, é o uso do módulo de kernel **virt_wifi**.

Funcionamento Técnico

Diferente do `mac80211_hwsim`, que simula toda a física do rádio, o `virt_wifi` atua como um "mascarador". Ele pega uma interface Ethernet existente (cabeada) e a apresenta ao sistema operacional como uma interface sem fio (`wlan0`), encapsulando os pacotes de forma transparente. Isso engana o Android fazendo-o acreditar que existe um adaptador Wi-Fi conectado, satisfazendo as verificações de HAL e permitindo que o `WifiManager` reporte uma conexão ativa.

Guia de Implementação e Comandos Críticos

A pesquisa recuperou scripts específicos discutidos pela comunidade para ativar essa funcionalidade. Note-se que estes comandos devem ser executados no **host** Linux antes ou durante a inicialização do contêiner, e não dentro da imagem Docker isoladamente.

Bash

```
# Carregar o módulo no kernel do host
sudo modprobe virt_wifi
```

```
# Derrubar a interface ethernet virtual que seria usada pelo container
# (Assumindo que enp0s1 é a interface alvo ou a veth do container)
sudo ip link set enp0s1 down
```

```
# Renomear a interface para um nome intermediário
sudo ip link set enp0s1 name wifi_eth
```

```
# Levantar a interface renomeada
sudo ip link set wifi_eth up
```

```
# Criar o link virtual Wi-Fi associado à interface Ethernet
sudo ip link add link wifi_eth name wlan0 type virt_wifi
```

O Paradoxo da Conectividade ADB

A aplicação dessa técnica introduz um efeito colateral crítico documentado nos relatórios de problemas: a perda de acesso ADB (Android Debug Bridge). O ADB daemon (`adbd`) dentro do Android é configurado para escutar e responder na interface de rede primária. Ao manipular as interfaces de rede para transformar `eth0` em `wlan0` via `virt_wifi`, a rota padrão e os bindings de porta do Docker (ex: `-p 5555:5555`) podem ser quebrados. O Android pode

assumir que, por ser uma interface Wi-Fi, ela não é a interface de depuração padrão (que seria USB ou Ethernet), ou o Docker pode perder a referência para o encaminhamento de portas NAT.³

A solução para este impasse envolve configurações avançadas de roteamento IP dentro do contêiner e, possivelmente, a alteração de propriedades do sistema (setprop service.adb.tcp.port 5555) para forçar o ADB a escutar em todas as interfaces, independentemente do tipo.

4. Solução 2: Simulação via Instrumentação Dinâmica (A Abordagem Recomendada)

Dada a complexidade e os riscos de instabilidade da modificação de kernel, a abordagem predominante na comunidade redroid para "simular" Wi-Fi ou LTE é a modificação em nível de aplicação (User-Space). Esta abordagem não exige que o kernel do host seja modificado, tornando-a compatível com a maioria das instalações Docker, incluindo aquelas em nuvens públicas onde o carregamento de módulos de kernel personalizados não é permitido.

4.1 O Ecossistema Magisk e Zygisk

Para que a simulação via software funcione, a imagem do redroid deve possuir acesso **root** gerenciado pelo **Magisk**. O Magisk permite a modificação do sistema sem alterar a partição de sistema (systemless interface). Mais importante, ele fornece o suporte ao **Zygisk** (evolução do Riru), que permite injetar código no processo Zygote. Como todos os aplicativos Android são "filhos" (forks) do processo Zygote, qualquer código injetado ali pode interceptar e modificar chamadas de API em todos os aplicativos do sistema.

4.2 Frameworks de Hooking: LSPosed

Sobre o Magisk/Zygisk, executa-se o **LSPosed**, que é a implementação moderna do Xposed Framework. O LSPosed gerencia módulos que contêm a lógica específica de simulação.⁶

Módulos de Simulação de Wi-Fi ("Fake Wi-Fi")

Módulos como "Fake Wifi" (desenvolvido originalmente por lemonsqueeze e mantido por chylek) operam interceptando chamadas para a classe android.net.wifi.WifiManager.⁴

Mecanismo de Ação:

1. **Interceptação:** O módulo "gancha" (hooks) métodos como getConnectionInfo(), getScanResults(), e getType() da classe NetworkInfo.
2. **Falsificação:** Quando um aplicativo alvo (ex: um jogo) chama esses métodos, o módulo bloqueia a resposta original do sistema (que diria "Ethernet" ou "Sem Conexão") e

retorna um objeto construído artificialmente contendo:

- SSID: "FakeWiFi" (ou nome customizado).
- BSSID: Um endereço MAC aleatório válido.
- SupplimentState: COMPLETED.
- DetailedState: CONNECTED.

3. **Transparência:** O tráfego de dados real continua fluindo pela interface eth0 do redroid, mas a lógica da aplicação é satisfeita pela informação falsa de que o transporte é Wi-Fi.

Módulos de Simulação de LTE ("Fake LTE/4G")

Para a simulação de dados móveis (LTE), utilizam-se módulos similares ou configurações avançadas de módulos de "Fake Network". Ferramentas como o **MobileIconChanger** ou funcionalidades dentro de toolkits como **Firefids Kit** permitem alterar a percepção do tipo de rede.⁹

Mecanismo de Ação:

1. **TelephonyManager:** O foco da interceptação muda para a classe `android.telephony.TelephonyManager`.
2. **NetworkType:** O módulo força o retorno de constantes como `NETWORK_TYPE_LTE` (13) ou `NETWORK_TYPE_NR` (20 - para 5G) quando o método `getNetworkType()` é invocado.
3. **Simulação de Sinal:** Alguns módulos avançados também permitem simular a força do sinal (Signal Strength) em dBm, o que é útil para testar o comportamento de apps em condições de sinal fraco, embora isso seja apenas uma simulação de metadados, não afetando a taxa de transferência real a menos que combinado com ferramentas de limitação de banda (network throttling) externas.²

5. Análise de Imagens Disponíveis e Scripts de Construção

Respondendo diretamente à questão sobre a existência de imagens: **Não existem imagens oficiais no Docker Hub mantidas pelo projeto remote-android que venham com Magisk, LSPosed e módulos de simulação pré-instalados.** A política do projeto oficial é fornecer imagens limpas (vanilla) do AOSP.

No entanto, a comunidade desenvolveu robustos *scripts de construção* que permitem a qualquer usuário gerar essas imagens personalizadas em minutos.

5.1 O Script redroid-script

A ferramenta mais citada e eficaz para gerar a imagem desejada é o repositório **redroid-script** (associado a usuários como abing7k ou versões fork de ayasa520).¹² Este script atua como um *wrapper* que:

1. Baixa a imagem oficial do redroid.
2. Monta o sistema de arquivos da imagem.
3. Injeta os binários do **Magisk**.
4. Injeta as bibliotecas **OpenGApps** (Google Play Services, essenciais para muitos apps que pedem Wi-Fi/LTE).
5. Injeta bibliotecas de tradução ARM (libndk) para rodar apps ARM em servidores x86.

Comando de Construção Típico:

A pesquisa identificou a sintaxe padrão utilizada por estes scripts para gerar uma imagem "rica":

Bash

```
# Exemplo de comando para gerar imagem Android 11 com GApps, Magisk e Widevine  
python redroid.py -a 11.0.0 -gmnw
```

- -a 11.0.0: Versão do Android (Android 11 é altamente recomendado para compatibilidade com módulos Xposed).
- -m: Instala Magisk (Habilita Root).
- -g: Instala GApps (Play Store).

O resultado é uma nova imagem Docker local (ex: redroid/redroid:a11_magisk_gapps) que possui a infraestrutura necessária para instalar os módulos de simulação.

5.2 Fluxo de Trabalho para Obter a Simulação

Uma vez gerada e executada a imagem com Magisk, o usuário deve seguir um fluxo manual para ativar a simulação, pois os módulos Xposed raramente vêm pré-ativados:

1. Acessar o Android via ADB ou Scrcpy.
2. Abrir o aplicativo Magisk e completar a configuração inicial.
3. Ativar o **Zygisk** nas configurações do Magisk e reiniciar o contêiner.
4. Instalar o **LSPosed** (ZIP) via Magisk e reiniciar.
5. Instalar o APK do **Fake Wifi** e ativá-lo na interface do LSPosed.

Esta é a resposta prática para a solicitação do usuário: a imagem não "existe" pronta para download público imediato (devido a licenças de GApps e complexidade), mas é trivialmente "fabricável" usando as ferramentas da comunidade.

6. Comparativo Técnico das Abordagens

A tabela abaixo sintetiza as descobertas, contrastando as duas metodologias identificadas para auxiliar na tomada de decisão técnica.

Característica	Simulação Nativa (Kernel - virt_wifi)	Simulação via Hooking (LSPosed - Fake Wifi)
Mecanismo	Cria interface de rede wlan0 real no Linux.	Intercepta chamadas de API Java e falsifica retorno.
Pré-requisitos	Módulos de Kernel no Host (virt_wifi.ko).	Magisk e LSPosed instalados no Android.
Dependência do Host	Alta (Exige acesso root no host e kernel compatível).	Baixa (Funciona em qualquer host Docker padrão).
Complexidade	Alta (Scripts de rede, namespaces, risco de ADB).	Média (Configuração inicial de módulos).
Fidelidade	Alta (Engana até verificações de baixo nível/HAL).	Média/Alta (Suficiente para 99% dos aplicativos).
Simulação LTE	Muito difícil (Exige emulação complexa de RIL).	Fácil (Via módulos que alteram TelephonyManager).
Caso de Uso Ideal	Desenvolvimento de sistema, testes de driver.	Jogos, Streaming, Burlar restrições de app.

7. Considerações Finais e Infraestrutura

A pesquisa conclui que, para a vasta maioria dos casos de uso — como execução de jogos que exigem Wi-Fi ou aplicativos de streaming que validam a conexão LTE — a abordagem de **Instrumentação Dinâmica (Magisk + LSPosed)** é superior. Ela contorna a necessidade de manipulação arriscada do kernel do host e oferece flexibilidade granular (ativar simulação por

aplicativo).

Para a simulação de LTE especificamente, a abordagem de kernel é inviável na arquitetura redroid atual, restando exclusivamente a opção de API Hooking. Módulos como **MobileIconChanger** e **Fake Connection** preenchem essa lacuna eficazmente.

Recomendações de Implementação

Recomenda-se a utilização de uma base **Android 11**, pois apresenta a maior estabilidade com o ecossistema Magisk/LSPosed atual. Versões mais recentes (Android 12/13/14) introduziram mecanismos de segurança e mudanças na estrutura de conectividade que podem dificultar a injeção de código necessária para os módulos de simulação funcionarem corretamente. Além disso, ao utilizar provedores de nuvem como Oracle Cloud (processadores ARM/Ampere), deve-se atentar para usar as variantes arm64 das imagens e módulos, garantindo compatibilidade binária.

Em suma, embora não exista uma imagem "plug-and-play" oficial, o ecossistema redroid fornece todas as ferramentas necessárias para construir uma solução personalizada robusta de simulação de rede, desde que o operador esteja disposto a realizar a etapa de construção da imagem e configuração dos módulos de *runtime*.

8. Detalhamento Técnico da Simulação de LTE (4G/5G)

A solicitação original especificou interesse tanto em Wi-Fi quanto em **LTE**. A seção a seguir aprofunda-se especificamente nas nuances da simulação de redes celulares, que difere significativamente da simulação Wi-Fi devido à ausência de hardware de rádio.

8.1 Limitações Físicas da Simulação LTE em Contêineres

Ao contrário do Wi-Fi, onde uma interface de rede pode ser facilmente "pontuada" ou simulada como uma placa wireless, a tecnologia LTE depende de um modem baseband (RIL - Radio Interface Layer) que comunica através de portas seriais ou barramentos proprietários com um chip de rádio. Em um servidor Docker (seja x86 ou ARM), esse hardware inexistente.

- **Emuladores vs. Redroid:** O emulador oficial do Android Studio simula o hardware do modem (RIL Daemon emulado) e permite comandos como `gsm signal 4` ou `network speed lte` via console telnet. O redroid, por design, remove ou simplifica a RIL para reduzir o *overhead*, resultando em um dispositivo que se reporta como "sem chip" ou "somente Wi-Fi/Ethernet".

8.2 Simulação via Propriedades de Sistema (build.prop)

Uma forma rudimentar de simular a *presença* de capacidade LTE é a manipulação das

propriedades de construção (build.prop) na imagem do redroid.

- **Habilitar RIL:** Algumas compilações do redroid possuem flags como `androidboot.redroid_radio=1` (padrão é 0/desativado).⁵ Ativar essa flag tenta carregar a pilha de telefonia.
- **Efeito:** Isso pode fazer com que as configurações de "Rede Móvel" apareçam no menu do Android, mas sem um daemon RIL funcional por trás para "conectar" a uma torre, o sistema ficará em estado de "Sem Serviço" ou "Emergência".

8.3 Simulação Eficaz via Xposed (Fake Network Type)

Para que um aplicativo acredite estar em LTE, a interceptação deve ocorrer na camada de aplicação, ignorando a falta de hardware.

- **Módulo "Fake Network" / "Hack Connectivity Service":** Estes módulos são essenciais para a simulação LTE. Eles operam monitorando as chamadas para `ConnectivityManager.getActiveNetworkInfo()`.
- **Substituição de Dados:** Quando a chamada é detectada, o módulo substitui o tipo de rede `TYPE_ETHERNET` ou `TYPE_WIFI` por `TYPE_MOBILE`.
- **Subtipo de Rede:** Crucialmente, eles permitem definir o subtipo para `TELEPHONY_NETWORK_TYPE_LTE` (4G) ou NR (5G). Isso satisfaz aplicativos que, por exemplo, limitam a qualidade de vídeo em redes móveis (para testar economia de dados) ou, inversamente, exigem rede móvel para autenticação de operadora (embora a autenticação real falhe sem o chip, a verificação de *tipo de rede* passa).

8.4 Ferramentas de Throttling (Limitação de Banda)

Para uma simulação LTE completa, não basta apenas mudar o ícone ou o nome da rede; é preciso simular as condições da rede (latência, perda de pacotes).

- **Traffic Control (tc):** No host Linux, ferramentas como `tc` (parte do pacote `iproute2`) podem ser usadas na interface virtual do contêiner (`veth`) para adicionar *delay* (ex: 50ms para simular 4G) e limitar a banda (ex: 20Mbps).
- **Integração:** Scripts de inicialização podem combinar a ativação do módulo Xposed (para a "mentira" visual/API) com regras `tc` no host (para a realidade física da conexão), criando um ambiente de teste LTE extremamente fiel para desenvolvedores.

Bash

```
# Exemplo teórico de limitação de banda no Host para simular LTE
# Adiciona 50ms de latência e limita a 20mbit na interface do container
tc qdisc add dev veth_redroid root tbf rate 20mbit burst 32kbit latency 50ms
```

Esta combinação de **Xposed (para a API)** e **Linux TC (para a física)** representa o estado da arte na simulação de LTE dentro do ecossistema redroid, superando as limitações da falta de modem físico.

Referências citadas

1. How can I emulate 4G (LTE) network in Android SDK - Stack Overflow, acessado em janeiro 27, 2026,
<https://stackoverflow.com/questions/21059914/how-can-i-emulate-4g-lte-network-in-android-sdk>
2. how to simulate a real mobile internet connection within android emulator - Stack Overflow, acessado em janeiro 27, 2026,
<https://stackoverflow.com/questions/14739386/how-to-simulate-a-real-mobile-internet-connection-within-android-emulator>
3. Discuss ways to improve Virtual Wifi · Issue #791 · remote-android/redroid-doc · GitHub, acessado em janeiro 27, 2026,
<https://github.com/remote-android/redroid-doc/issues/791>
4. Fake Wifi (Xposed Module) - Apps on Google Play, acessado em janeiro 27, 2026,
https://play.google.com/store/apps/details?id=eu.chylek.adam.fakewifi&hl=en_US
5. redroid-rockchip - GitHub, acessado em janeiro 27, 2026,
<https://github.com/redroid-rockchip>
6. [Android] How to Spoof with LSposed Guide : r/PokemonGoSpoofing - Reddit, acessado em janeiro 27, 2026,
https://www.reddit.com/r/PokemonGoSpoofing/comments/1dsszs/android_how_to_spoof_with_lsposed_guide/
7. LSposed Framework - GitHub, acessado em janeiro 27, 2026,
<https://github.com/LSposed/LSposed>
8. chylek/FakeWifi: Make android apps believe Wifi is connected. Handy in situations where there is a (non wifi) connection, but some app won't do its thing unless it thinks it's on wifi. - GitHub, acessado em janeiro 27, 2026,
<https://github.com/chylek/FakeWifi>
9. programminghoch10/MobileIconChanger: Xposed module to change the mobile data icons - GitHub, acessado em janeiro 27, 2026,
<https://github.com/programminghoch10/MobileIconChanger>
10. Firefds/FirefdsKit: Xposed module for Samsung devices (Android 6, 7, 8, 9, 10, 11, 12, 13, 14) - GitHub, acessado em janeiro 27, 2026,
<https://github.com/Firefds/FirefdsKit>
11. How can I simulate low cellular signal at home to test my app on a real device? - Reddit, acessado em janeiro 27, 2026,
https://www.reddit.com/r/androiddev/comments/1nqw7i4/how_can_i_simulate_low_cellular_signal_at_home_to/
12. abing7k/redroid-script - GitHub, acessado em janeiro 27, 2026,
<https://github.com/abing7k/redroid-script>